

Evidencia de Ejecución — Semana 3

Automatización, pruebas y flujo CI/CD

Proyecto	NovaTareas Pro — Asistente inteligente de gestión de tareas
Módulo	Módulo 4 — Desarrollo de Aplicaciones con IA
Equipo	Equipo 3
Integrantes	Saúl Oswaldo López Hernández (SMIS108421) Moises Antonio Martínez (SMIS071221) Enson Onan Carranza Rodríguez (SMIS013020)
Docente	Ing. Marco Arévalo Zambrano

Resumen de la entrega

Elemento	Resultado
Pruebas automatizadas	25 pruebas, 100% superadas (Vitest)
Tiempo de ejecución	752 ms
Pipeline CI/CD	GitHub Actions — ejecución exitosa en 41 s
Verificación de tipos	57 errores preexistentes corregidos → 0
Compilación	Build completo en 4,15 s
Gestión de secretos	.env.example con placeholders; sin credenciales en el repositorio

1. Evidencia de pruebas ejecutadas localmente

Las pruebas se implementaron con **Vitest**, elegido sobre Jest por su compatibilidad nativa con módulos ESM y con Astro, el framework del proyecto. Se ejecutan con un solo comando y no requieren conexión a internet ni credenciales.

Comando ejecutado

```
$ npm test
```

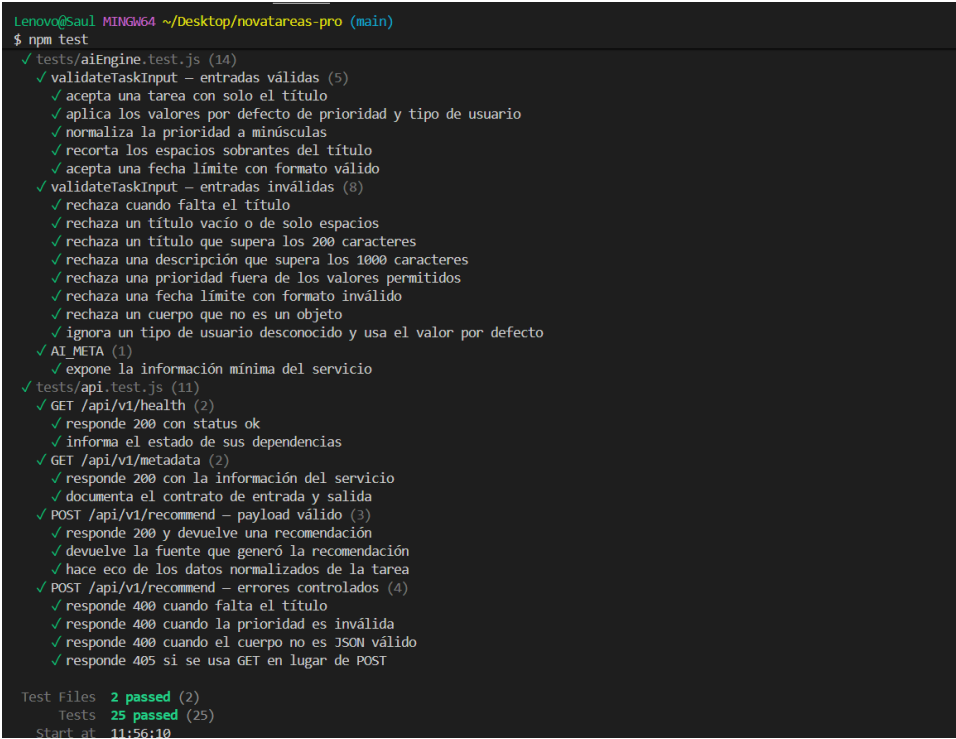


Figura 1. Salida de `npm test`: 2 archivos de prueba, 25 pruebas superadas.

Cobertura de las pruebas

Archivo	N.º	Comportamiento validado
tests/aiEngine.test.js	14	Validación de entrada: títulos vacíos o excesivos, descripciones fuera de límite, prioridades no permitidas, fechas mal formadas, cuerpos que no son objetos
tests/api.test.js	11	Salud del servicio, contrato de metadatos, respuestas exitosas del endpoint de IA y errores controlados (400 y 405)

Las pruebas de endpoints siguen el mismo principio que el *TestClient* de FastAPI visto en clase: en lugar de levantar el servidor y lanzar peticiones manuales con curl, importan directamente el manejador del endpoint y le pasan un objeto `Request`. Esto las hace rápidas, repetibles y ejecutables sin abrir ningún puerto.

2. Validación de tipos y compilación local

Además de las pruebas, el flujo incluye dos validaciones que se ejecutan igual en local y en el pipeline: la verificación de tipos con `astro check` y la compilación del proyecto. La verificación de tipos detectó **57 errores preexistentes** que fueron corregidos (ver sección 4).

```
Lenovo@Saul MINGW64 ~/Desktop/novatareas-pro (main)
$ npm run lint
33 'UPDATE users SET google_access_token = ?, google_refresh_token = ?, google_token_expiry = ? WHERE id = ?'
34 ).run(tokens.access_token || null, refreshToken, tokens.expiry_date ? tokens.expiry_date.toString() : null, user.userId);

src/pages/api/tasks/[id]/ai.js:55:22 - warning ts(80007): 'await' has no effect on the type of this expression.
55 const ragContext = await getRagContext(user.userId, task);

src/pages/api/tasks/[id]/ai.js:3:25 - warning ts(6133): 'reindexTask' is declared but its value is never read.
3 import { getRagContext, reindexTask } from '../../../lib/rag.js';

telegram/bot.js:3:7 - warning ts(6133): 'require' is declared but its value is never read.
3 const require = createRequire(import.meta.url);

tools/import_csv_and_embed.mjs:8:7 - warning ts(6133): 'require' is declared but its value is never read.
8 const require = createRequire(import.meta.url);

Result (56 files):
- 0 errors
- 0 warnings
- 31 hints
```

Figura 2. Salida de `npm run lint`: 0 errores y 0 advertencias. Los 31 elementos restantes son sugerencias informativas que no interrumpen la validación.

```
Lenovo@Saul MINGW64 ~/Desktop/novatareas-pro (main)
$ npm run build

> build
> astro build

07:59:30 [vite] Re-optimizing dependencies because vite config has changed
07:59:30 [types] Generated 178ms
07:59:30 [build] output: "server"
07:59:30 [build] directory: C:\Users\Lenovo\Desktop\novatareas-pro\dist\
07:59:30 [build] adapter: @astrojs/node
07:59:30 [build] Collecting build info...
07:59:30 [build] ✓ Completed in 210ms.
07:59:30 [build] Building server entrypoints...
07:59:33 [vite] ✓ built in 2.52s
07:59:33 [build] ✓ Completed in 2.75s.

building client (vite)
07:59:34 [vite] ✓ 29 modules transformed.
07:59:34 [vite] dist/client/_astro/hoisted.BuGMYses.js 0.16 kB | gzip: 0.11 kB
07:59:34 [vite] dist/client/_astro/hoisted.Befybygc.js 1.70 kB | gzip: 0.72 kB
07:59:34 [vite] dist/client/_astro/hoisted.ChkFSOEg.js 6.07 kB | gzip: 1.84 kB
07:59:34 [vite] dist/client/_astro/client.BRZKPEzt.js 142.41 kB | gzip: 45.92 kB
07:59:34 [vite] ✓ built in 911ms
07:59:34 [build] Rearranging server assets...
07:59:34 [build] Server built in 4.15s
07:59:34 [build] Complete!
```

Figura 3. Salida de `npm run build`: compilación completa del servidor y del cliente en 4,15 segundos.

3. Evidencia del pipeline CI/CD

El archivo `.github/workflows/ci.yml` define un flujo que se ejecuta automáticamente en GitHub Actions con cada `push` o `pull request`. El pipeline instala las dependencias, valida los tipos, ejecuta las pruebas y compila la aplicación.

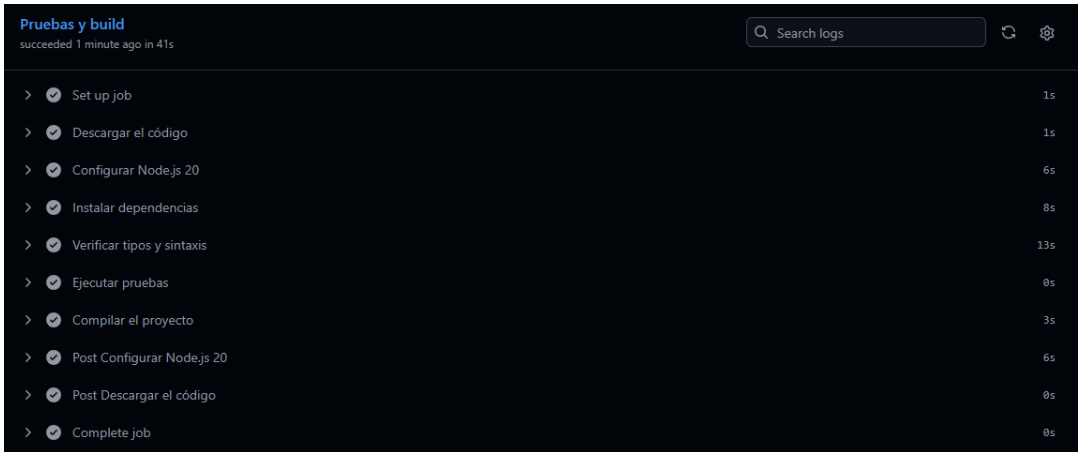


Figura 4. Ejecución del workflow «Pruebas y build» en GitHub Actions: todos los pasos completados correctamente en 41 segundos.

Pasos del pipeline

Pas o	Acción	Propósito
1	Descargar el código	Obtiene el repositorio en el runner
2	Configurar Node.js 20	Fija la versión del entorno y activa la caché de npm
3	Instalar dependencias	<code>npm ci</code> : instalación reproducible desde <code>package-lock.json</code>
4	Verificar tipos y sintaxis	<code>npm run lint</code>
5	Ejecutar pruebas	<code>npm test</code> : las 25 pruebas
6	Compilar el proyecto	<code>npm run build</code> : confirma que la aplicación se construye

Ningún paso admite fallos. En una versión inicial, la verificación de tipos estaba marcada como informativa porque el proyecto arrastraba errores previos. Una vez corregidos, esa excepción se eliminó: hoy cualquier error de tipos, prueba fallida o fallo de compilación detiene el pipeline.

Gestión de secretos en el pipeline

El workflow **no expone la variable** `ZAI_API_KEY` de forma intencional. Sin esa clave, el motor de IA recurre a su cascada de respaldo y responde con reglas locales, lo que hace que las pruebas sean deterministas, no consuman saldo de la API en cada ejecución y no fallen por problemas de red. Sí se inyectan `SECRET_KEY` y `CRON_SECRET` con valores ficticios, porque el módulo de autenticación interrumpe el arranque si no están definidos.

4. Errores detectados y corregidos

Los dos casos siguientes se detectaron precisamente **gracias a automatizar la validación**. Ninguno era visible ejecutando la aplicación a mano.

4.1. Las pruebas llamaban a la API real y agotaban el tiempo de espera

```
■ POST /api/v1/recommend – payload valido (3) 30165ms
  x responde 200 y devuelve una recomendacion 10017ms
  x devuelve la fuente que genero la recomendacion 10142ms
  x hace eco de los datos normalizados de la tarea 10005ms

Error: Test timed out in 10000ms.

Test Files 1 failed | 1 passed (2)
  Tests 3 failed | 22 passed (25)
Duration 33.42s
```

Figura 5. Primera ejecución de la suite: 3 pruebas fallidas por tiempo de espera agotado.

Diagnóstico. Las tres pruebas que fallaron son las que invocan la generación de recomendaciones. Se comprobó que Vitest carga automáticamente el archivo `.env` del proyecto, por lo que la clave de la API sí estaba definida durante la ejecución. El motor intentaba contactar el servicio de IA por internet con un tiempo de espera de 45 segundos, mientras que el límite por prueba era de 10.

Corrección. Se añadió un bloque `env` en `vitest.config.js` que vacía la clave de la API y apunta el servicio local a un puerto cerrado, de modo que el motor recurre de inmediato a sus reglas locales. Se define en el archivo de configuración y no dentro de las pruebas, porque el módulo lee esas variables al cargarse, antes de que el test pueda modificarlas.

Indicador	Antes	Después
Pruebas superadas	22 de 25	25 de 25
Duración de la suite	33,42 s	752 ms
Dependencia de red	Sí	No
Consumo de saldo de la API	Sí, en cada ejecución	Ninguno

4.2. Cincuenta y siete errores de tipado sin detectar

Síntoma. Al incorporar la verificación de tipos al pipeline, astro check reportó 57 errores repartidos en tres archivos de página. La aplicación funcionaba con normalidad, porque los tipos se eliminan al compilar y el código que se ejecuta es JavaScript.

Diagnóstico. Los errores respondían a tres patrones: consultas a la base de datos cuyo resultado se declaraba como desconocido, elementos del DOM tratados como genéricos en lugar de campos de formulario, y funciones asignadas al objeto global sin declarar. Además, el proyecto no tenía archivo `tsconfig.json`, por lo que la verificación corría con la configuración más estricta disponible.

Corrección. Se añadieron afirmaciones de tipo en los tres archivos y un `tsconfig.json` basado en la configuración oficial de Astro. Ningún cambio altera el comportamiento en ejecución. Durante el proceso se descartó una corrección inicial que tipaba los parámetros de los manejadores de eventos, porque generaba errores nuevos al acceder a los campos del formulario.

Indicador	Antes	Después
Errores de tipado	57	0
Archivos con errores	3	0
Paso de validación en el pipeline	Informativo (no bloqueaba)	Bloqueante